



DEUCENG

Dokuz Eylül University
Dept of Computer Engineering



**CME2210 – Object Oriented
Analysis and Design**

Term Project-Final Report

Barber Appointment System

Prepared by Abdülkadir ERTAŞ

May 2026

Contents

1. INTRODUCTION.....	5
1.1 Definitions, Acronyms, and Abbreviations	5
1.2 Overview.....	6
2. PROBLEM DEFINITION	8
3. REQUIREMENTS	9
3.1 Key Use Cases.....	9
3.1.1 UC-1: Register/Apply to System	9
3.1.2 UC-2: Approve Applications	10
3.1.3 UC-3: Book an Appointment.....	11
3.1.4 UC-4: View Appointment Details	13
3.2 Functional Requirements.....	14
3.3 Non-Functional Requirements	15
4. ARCHITECTURAL DESIGN	17
4.1 Overall System Design Description	17
4.2 System Architecture	17
5. UML DIAGRAMS.....	20
5.1 Use Case Diagram	20
5.2 Class Diagrams	21
5.2.1 Overall System Class Diagram.....	21
5.2.2 Entity Layer	22
5.2.3 Infrastructure Layer	23
5.2.4 Presentation Layer	23
5.2.5 Core Application Layer	24
5.3 Activity Diagrams.....	25
5.3.1 Admin Application Approval	25
5.3.2 Appointment State Initialization Activity Diagram.....	26

5.4 Sequence Diagrams	27
5.4.1 Login Sequence Diagram	27
5.4.2 Book An Appointment Sequence Diagram	27
5.4.3 Display Barbers Schedule Sequence Diagram	28
5.4.4 Remove a User Sequence Diagram	28
5.5 Statechart Diagrams	29
5.5.1 Barber's Application Status Statechart Diagram	29
5.5.2 Appointment status Statechart Diagram	29
6. DATA FLOW DIAGRAMS (DFD)	30
6.1 Level 0 Context Diagram	30
6.2 Level 1 Data Flow Diagram: Appointment Booking.....	30
6.3 Level 1 Data Flow Diagram: User Enrollment.....	31
7. DESIGN PATTERN USED	32
7.1 Facade Pattern	32
7.2 Strategy Pattern	32
7.3 Builder Pattern	32
7.4 Factory Pattern	33
7.5 Singleton Pattern	33
8. SOLID EVALUATION	34
8.1 Single Responsibility Principle (SRP)	34
8.2 Open/Closed Principle (OCP)	35
8.3 Liskov Substitution Principle (LSP)	36
8.4 Interface Segregation Principle (ISP)	36
8.5 Dependency Inversion Principle (DIP)	37
9. TESTING STRATEGY	39
9.1 Unit Testing (Business Logic Isolation).....	39
9.2 Data Persistence and Integration Testing	39
9.3 Input Validation and Edge Case Handling	39
9.4 End-to-End (E2E) Workflow Testing	40

10. LIMITATIONS.....	41
10.1 Data Persistence and Concurrency	41
10.2 User Interface Constraints.....	41
10.3 Security and Session Management	42
10.4 Functional Scope Limitations	42
11. Reflection and Future Improvements.....	43
11.1 Reflection and Lessons Learned	43
11.2 Future Enhancements.....	43
12. CONCLUSION	44
13. REFERENCES	45
APPENDIX A: USER MANUEL & SYSTEM EXECUTION	46
A.1 Main Menu.....	46
A.2 Login and Customer Menu	47
A.3 Booking an Appointment: Barber Selection	48
A.4 Booking an Appointment: Schedule and Time Slot Selection	49
A.5 Booking an Appointment: Confirmation and Finalization	50
A.6 Viewing Active Appointments	51
APPENDIX B: Data Persistence AND CSV File Structure	52

1. INTRODUCTION

The Barber Appointment Scheduling System is a console-based Java application designed to manage reservations, user accounts, and daily schedules for a barbershop environment. The primary focus of this project is not only to deliver a functional scheduling tool but to demonstrate a strong command of object-oriented programming (OOP) principles and modern software architecture.

The system supports three distinct user roles: Customers, Barbers, and Administrators. Customers can book, view, and cancel appointments. Barbers can track their daily schedules and manage their active sessions. Administrators oversee the platform, handling the approval or rejection of new barber applications and maintaining overall system control.

Rather than relying on basic procedural logic, the project is structured around Clean Architecture guidelines. It strictly separates concerns into Presentation, Core Application, Infrastructure, and Entity layers. Throughout the development process, careful attention was given to implementing structural design patterns, manual dependency injection, and SOLID principles to ensure the codebase remains maintainable and extensible.

This document outlines the complete lifecycle and structural decisions behind the project. It details the functional and non-functional requirements, architectural design choices, testing strategies, and the technical trade-offs made to ensure data integrity and system stability.

1.1 Definitions, Acronyms, and Abbreviations

This subsection provides the definitions of all terms, acronyms, and abbreviations required to interpret the Software Architecture Document properly.

Acronyms and Abbreviations:

- **CSV:** Comma-Separated Values (The format used for data persistence in this system).
- **DFD:** Data Flow Diagram.
- **DTO:** Data Transfer Object.
- **GUI:** Graphical User Interface.
- **OOP:** Object-Oriented Programming.
- **SOLID:** An acronym for five core design principles (Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion).
- **UI:** User Interface.

- **UML:** Unified Modeling Language.

Definitions:

- **Entity:** A core business object that represents a real-world concept within the domain (e.g., Customer, Barber, Appointment).
- **Facade:** A structural design pattern that provides a simplified, unified interface to a complex subsystem.
- **Repository:** A structural layer responsible for encapsulating the logic required to access data sources (in this case, CSV files).
- **Dumb UI:** A design principle where the presentation layer contains no business logic and relies entirely on underlying services for decision-making.

1.2 Overview

This document provides a comprehensive architectural and technical blueprint of the Barber Appointment System. It details the software development lifecycle of the project, from initial requirements gathering and structural design to implementation strategies, testing, and future projections. The document is logically organized to guide the reader through the system's engineering principles:

- **Sections 2 and 3 (Problem Definition & Requirements):** Establish the foundational context by defining the core problem the system aims to solve. These sections outline the system's functional and non-functional requirements, alongside detailed descriptions of key use cases such as user registration, administrative approvals, and appointment management.
- **Section 4 (Architectural Design):** Presents the high-level system architecture, explaining the layered design approach that strictly separates the presentation interface, core application logic, and infrastructure components.
- **Section 5 (UML Diagrams):** Provides an exhaustive visual representation of the system. It utilizes Use Case, Class, Activity, Sequence, and Statechart diagrams to illustrate both the static object-oriented structure and the dynamic behavioral workflows of the application.
- **Section 6 (Data Flow Diagrams):** Complements the UML models by mapping the logical movement of information. Through Level 0 and Level 1 DFDs, it visualizes how data interacts with external entities and data stores during critical processes like appointment booking and user enrollment.

- **Sections 7 and 8 (Design Patterns & SOLID Evaluation):** Articulate the theoretical foundation of the codebase. Section 7 details the implementation of structural, behavioral, and creational design patterns (Facade, Strategy, Builder, Factory, and Singleton). Section 8 provides a critical evaluation of how the system tends to the five SOLID principles to ensure maintainability and scalability.
- **Section 9 (Testing Strategy):** Outlines the comprehensive testing approach employed to verify system reliability, encompassing business logic isolation (unit testing), data persistence integration, edge case handling, and end-to-end workflow validation.
- **Sections 10 and 11 (Limitations & Future Improvements):** Provide a transparent technical assessment of the system. These sections identify current constraints regarding concurrency, UI, and functional scope, followed by reflections on the development process and proposed enhancements for future iterations.
- **Sections 12 and 13 (Conclusion & References):** Summarize the overarching achievements of the project and list the academic or technical resources consulted during the development of this architecture.
- **Appendix A:** User Manual and System Execution
- **Appendix B:** Data Persistence and CSV File Structure

2. PROBLEM DEFINITION

The traditional method of managing barbershop appointments often relies on phone calls, physical notebooks, or simple messaging. This manual approach tends to human error, such as double-booking time slots, forgetting appointments, and losing track of customer history.

From a software engineering perspective, digitizing this process introduces several technical challenges that must be addressed to create a reliable and consistent system. The primary problems this project aims to solve include:

- **Schedule Collisions:** Ensuring that a single barber cannot be booked by multiple customers for the same time slot. The system must validate availability in real-time before confirming any appointment.
- **Data Integrity and Synchronization:** Appointments inherently belong to both the customer and the barber. A major technical challenge is maintaining a bidirectional relationship. For instance, if a barber is removed from the system by an administrator, the system must perform a cascading update to cancel all future appointments tied to that barber, preventing "orphan" records.
- **Role-Based Access and Logic:** Different users (Admin, Barber, Customer) require completely different interfaces, permissions, and data views. The system must securely authenticate users and restrict their actions to their authorized scope (e.g., a customer cannot view another customer's appointments, and only admins can approve new barbers).
- **Data Persistence:** Since the application uses CSV files instead of a relational database, reading, updating, and saving state changes (like marking past appointments as "PAST" or updating user statuses) must be handled carefully to prevent file corruption or data loss between sessions.

In short, the project solves the practical problem of barbershop scheduling while addressing the underlying computer science problems of data consistency, state management, and architectural separation of concerns.

3. REQUIREMENTS

This section defines the exact specifications and constraints that shape the Barber Appointment Scheduling System. It is categorized into three main areas: Use Cases, Functional Requirements (FRs), and Non-Functional Requirements (NFRs). The Use Cases map out the expected interactions between the system and its three primary actors (Admin, Barber, and Customer). The Functional Requirements detail the core operations, business rules, and data processing tasks the software must execute. Finally, the Non-Functional Requirements specify the system's quality attributes, focusing on architectural constraints, data integrity, and execution environment standards.

3.1 Key Use Cases

3.1.1 UC-1: Register/Apply to System

Use Case Name: Register/Apply to the System

Purpose: Enrollment of the user

Description: Enroll users into system by getting personal information. If the user is a barber, it is required to approve the application by the Admin.

Actors: Customer, Barber

Data Content: User information (Name, Surname, Phone Number, Password/PIN). For Barbers additionally: Shop Name.

Pre-Condition:

- The user must be logged out (Guest).

Begins When: When “Sign Up as Customer” or “Apply as Barber” menu is triggered.

Ends When: When the user confirms the application and the system saves the record.

Exceptions:

- Exception-1: (Already Singed in): If the user has already signed in, system displays an alert message (“You have already signed in. You will be redirected to the Main Menu.”), and the user is redirected to Main Menu.
- Exception-2: (Invalid or Empty Input): If the user enters invalid input while fully filling the information, system displays an alert message (“Invalid input. Please try again.”) and asks for rewrite new input.

- **Exception-3: (Duplicate Phone Number):** If the user enters a phone number that already exists in the database, the system displays an alert ("This phone number is already registered") and asks for a different input.

Post-Condition: The user is saved to database (CSV file). If the user is barber, state is marked as “PENDING”.

References: FR-3.2.1, FR-3.2.15

Main Flow:

Table 1: Main Flow of Use Case 1 (Register/Apply)

	Actions Performed by the Actor		Responses from the System
1-	Select Register/Apply Menu.		
		2-	Ask for personal information.
3-	Enter personal details.		
		4-	Display personal details entered by the user. Ask for confirmation.
5-	Confirm information.		
		6-	Save user to database (CSV) and display confirmation message.

3.1.2 UC-2: Approve Applications

Use Case Name: Approve Applications

Purpose: Approval of pending applications.

Description: Barber applications need approval to use the system. Pending applications are approved by the Admin.

Actors: Admin

Data Content: Barber information (Barber ID, Barber Name, Phone Number, Shop Name, Application Status).

Pre-Condition:

- The user must be logged in as Admin.

Begins When: When “Approve Applications” menu is triggered by the Admin.

Ends When: When the Admin finishes the approval process and enters the escape code ("0") to return to the Main Menu.

Exceptions:

- Exception-1:(Empty Application List): If there is no application, system displays an alert message ("There is no pending application."), and the user is redirected to the Main Menu.

Post-Condition: The selected barber(s) are approved, and their status in the database (CSV file) is updated from "PENDING" to "APPROVED".

References: FR-3.2.5, FR-3.2.6, FR-3.2.15

Main Flow:

Table 2: Main Flow of Use Case 2 (Approve Applications)

	Actions Performed by the Actor		Responses from the System
1-	Select "View Applications" menu.		
		2-	Retrieve and display the list of pending applications.
3-	Select an application to approve		
		4-	Update barber's application status to "APPROVED" and display confirmation message.
5-	Enter escape code ("0").		
		6-	Redirect the user to the Main Menu.

3.1.3 UC-3: Book an Appointment

Use Case Name: Book an Appointment

Purpose: Booking an appointment for the customer.

Description: The main purpose of the system. At this case, customer selects desired barber, time slot, service and creates an appointment.

Actors: Customer

Data Content:

- Approved barbers list.
- Appointment details (Barber Name, Customer information, Selected Date/Time).

Pre-Condition:

- The user must be logged in as Customer.

Begins When: When “Book an Appointment” menu is triggered by the Customer.

Ends When: When the user confirms the appointment and the system saves the record.

Exceptions:

- Exception-1: (Invalid or Empty Input): If the user enters invalid or empty input, the system displays an alert message (“Invalid input. Please try again.”) and asks for rewrite new input.
- Exception-2: (Selecting Occupied Slot): If the user selects a time slot which is not available, the system displays an alert message (“This time slot has already taken. Please select an available slot.”) and asks for enter a new time slot.

Post-Condition: The appointment is saved to database, and the selected time slot's availability status is updated as "FALSE".

References: FR-3.2.7, FR-3.2.8, FR-3.2.9, FR-3.2.10, FR-3.2.15

Main Flow:

Table 3: Main Flow of Use Case 3 (Book an Appointment)

	Actions Performed by the Actor		Responses from the System
1-	Select “Book an Appointment” menu.		
		2-	Retrieve and display the list of the approved barbers.
3-	Select a barber from the list.		
		4-	Display daily schedule for the selected barber, mark each slot as “Available” or “Occupied”.
5-	Enter the desired time slot which is marked as “Available”.		
		6-	Verifying the selection is valid. Display the appointment details. Ask to save the appointment.
7-	Confirm appointment.		
		8-	Save appointment. Update time slots and display confirmation message.

3.1.4 UC-4: View Appointment Details

Use Case Name: View Appointment Details

Purpose: To allow users to view their scheduled appointments.

Description: Informs the user about the appointments. Includes minor differences for the barbers and customers. The customers view their current appointments; the barbers view the list of the customers who booked an appointment.

Actors: Customer, Barber.

Data Content: List of the appointments (Date, Time, Service, Counterpart Information).

Pre-Condition:

- The user must be logged in.

Begins When: When “View Active Appointment Details” menu is triggered by the user.

Ends When: When the user enters escape code to return to the Main Menu.

Exceptions:

- Exception-1: (Empty Appointment List): If the user does not have any appointments, the system displays an alert message (“There are no active appointments.”).

Post-Condition: None. System state remains unchanged.

References: FR-3.2.13, FR-3.2.14, FR-3.2.15

Main Flow:

Table 4: Main Flow of Use Case 4 (View Appointment Details)

	Actions Performed by the Actor		Responses from the System
1-	Select “View Appointment Details” menu.		
		2-	Identify the actor type (Customer/Barber). Retrieving the appointments from the database (CSV) matching the user’s phone number. Display appointment details.
3-	Enter escape code (“0”).		
		4-	Redirect the user to the Main Menu.

3.2 Functional Requirements

User & Authentication Management

- **FR-3.2.1:** The system shall allow users (Customers and Barbers) to register by providing their Name (also shop name for barbers), Surname, Phone Number, and Password/PIN.
- **FR-3.2.2:** The system shall allow users and the Admin to log in using their registered Phone Number and Password/PIN.
- **FR-3.2.3:** The system shall prevent users with "PENDING" status (unapproved barbers) from accessing the main appointment features.
- **FR-3.2.4:** The system shall allow users to log out safely and return to the main menu.

Administrative Functions

- **FR-3.2.5:** The system shall allow the Admin to view a list of all pending barber applications.
- **FR-3.2.6:** The system shall allow the Admin to approve pending barber applications, updating their status to "APPROVED" in the database.
- **FR-3.2.6.1:** The system shall allow the Admin to reject pending barber applications, which subsequently removes the rejected barber's account from the system.
- **FR-3.2.6.2:** The system shall notify Barbers of their application status (Pending or Rejected) upon their login attempt before being approved.

Appointment Management

- **FR-3.2.7:** The system shall display a list of all "APPROVED" barbers to the Customer.
- **FR-3.2.8:** The system shall display the daily schedule of a selected barber, clearly marking time slots as "Available" or "Booked".
- **FR-3.2.9:** The system shall allow Customers to book an available time slot for a selected barber.
- **FR-3.2.10:** The system shall verify slot availability in real-time to prevent double-booking.
- **FR-3.2.11:** The system shall allow Customers and Barbers to cancel their upcoming appointments.
- **FR-3.2.12:** The system shall automatically update a canceled time slot's status back to "Available".

Data Viewing & Reporting

- **FR-3.2.13:** The system shall allow Customers to view the details of their upcoming appointments (Date, Time, Barber Name, Service).
- **FR-3.2.14:** The system shall allow Barbers to view their booked appointments and the details of the Customers who booked them.
- **FR-3.2.15:** The system shall read from CSV and write to CSV files persistently to ensure data is not lost when the program closes.
- **FR-3.2.16:** The system shall allow users to deactivate/remove their own accounts and allow the Admin to remove any user account.
- **FR-3.2.17:** The system shall perform a cascading update when a user account is removed, automatically changing the status of all associated "ACTIVE" appointments to "CANCELLED".

Automatic Appointment Status Update

- **FR-3.2.18:** The system shall automatically detect and update the status of past "ACTIVE" appointments to "PAST" during the data loading phase based on the current system time.

3.3 Non-Functional Requirements

Performance

- **NFR-3.3.1:** The system shall process user inputs and display lists (e.g., available time slots, approved barbers) in less than 2 seconds.

Design Constraints

- **NFR-3.3.2:** The system shall be developed strictly using the Java programming language and utilize CSV files for persistent data storage.

Usability

- **NFR-3.3.3:** The system shall provide a clear, easy-to-navigate Command Line Interface (CLI) using numbered menu options.
- **NFR-3.3.4:** The system shall provide immediate, standardized feedback messages (e.g., "SUCCESS", "ERROR") for all user actions.

Reliability & Data Integrity

- **NFR-3.3.5:** The system shall prevent data loss by immediately saving state changes (registrations, appointments, status updates) to the CSV database.

- **NFR-3.3.6:** The system shall ensure data integrity by maintaining synchronized bidirectional relationships in memory (e.g., updating both Customer and Barber lists when an appointment is booked).

Security

- **NFR-3.3.7:** The system shall restrict administrative functions strictly to the logged-in Admin account.
- **NFR-3.3.8:** The system shall securely hash all user passwords (e.g., using SHA-256 algorithm) before persisting them to the database, ensuring raw passwords are never exposed.

Maintainability & Architecture

- **NFR-3.3.9:** The system architecture shall obey Clean Architecture principles, ensuring a strict separation between Presentation, Core Application, Infrastructure, and Entity layers.
- **NFR-3.3.10:** The system shall utilize manual Dependency Injection to minimize tight coupling between components and facilitate unit testing.

4. ARCHITECTURAL DESIGN

4.1 Overall System Design Description

The Barber Appointment System is designed with a strong emphasis on maintainability, scalability, and loose coupling, strictly adhering to Clean Architecture and Object-Oriented Programming (OOP) principles. To achieve a highly modular structure, the system utilizes a Layered Architecture approach (packaged as Presentation, Core Application, Infrastructure, and Entity), which cleanly separates the user interface, business rules, and data management into distinct boundaries.

The overall design philosophy is driven by the following core architectural decisions:

- **Separation of Concerns (SoC):** The system is logically divided into primary layers. This separation ensures that a change in one layer (e.g., modifying the console interface or changing the file storage format) does not affect the core business rules or the underlying data entities.
- **SOLID Principles & Dependency Inversion:** The design relies heavily on SOLID principles. Specifically, the Dependency Inversion Principle (DIP) is implemented using the `IRepository<T>` interface. Service classes do not depend on concrete data storage classes but rather on abstractions.
- **Manual Dependency Injection (DI):** To avoid tight coupling, dependencies (such as repositories and services) are not instantiated within the classes that use them. Instead, they are created at the application root (Main class) and injected manually via constructors. This makes the system highly testable.
- **Facade Pattern for UI Isolation:** The system introduces an `AppointmentSystemFacade` to act as a structural shield between the Presentation layer and the Core Application layer. The "Dumb UI" only interacts with this Facade, completely isolating it from the complexities of the underlying service interactions.
- **Rich Domain Model:** Instead of using anemic data structures, the system employs a Rich Domain Model. Entities such as Barber, Customer, and Appointment encapsulate their own specific behaviors and states.

4.2 System Architecture

The system architecture is structured around a strict layered approach. Communication flows from the Presentation Layer downwards, utilizing Data Transfer Objects (DTOs) to prevent exposing raw entity data.

1. Presentation Layer

- **Component:** ConsoleUI
- **Responsibility:** This layer acts as the interaction point for the actors. It is strictly responsible for capturing user input and displaying output messages.
- **Architectural Rule:** It contains zero business logic. It delegates all operations to the AppointmentSystemFacade. It sends primitive data types to the Facade and receives DTOs (AppointmentResponseDTO, AuthResponseDTO) in return, ensuring a secure "Dumb UI" implementation.

2. Core Application Layer (Service Layer) This is the brain of the system, encapsulate all business rules, validations, and workflow orchestrations.

- **AppointmentSystemFacade:** Orchestrates the interactions between the UI and the individual services, providing a simplified, unified interface for the presentation layer.
- **AuthService:** Manages user authentication, session state, and user removal. Instead of a basic utility class, it utilizes the Strategy Pattern (IHashStrategy, Sha256HashStrategy) for secure, interchangeable password hashing logic.
- **AppointmentService:** Handles the core domain logic regarding appointments. It checks schedule availability, prevents double-booking, and manages the bidirectional data synchronization (updating both Barber and Customer appointment histories simultaneously).
- **ApplicationService:** Dedicated to the Admin workflow for reviewing, approving, or rejecting new Barber applications.

3. Infrastructure Layer (Data Access)

- **Components:** IRepository<T> (Interface), IUserRepository<T>(Interface), CustomerRepository, BarberRepository, AppointmentRepository, and RepositoryFactory.
- **Responsibility:** This layer is completely responsible for data persistence (reading from and writing to CSV files) and loading entities into memory during system startup.
- **Architectural Rule:** By utilizing the generic interfaces and a RepositoryFactory for creation, the Service Layer remains completely decoupled from the file system.

4. Entity Layer (Rich Domain Model)

- **Components:** User, Customer, Barber, Appointment and related Enums (AppointmentState, ApplicationState).
- **Responsibility:** These entities are the core of the application. They strictly encapsulate their own specific business behaviors. For instance, the Barber entity contains its own schedule map and a List<Appointment> history, exposing specific methods (updateSchedule) to modify them safely. This strict encapsulation prevents the Service Layer from micromanaging entity-level data, fully adhering to OOP principles.

5. UML DIAGRAMS

5.1 Use Case Diagram

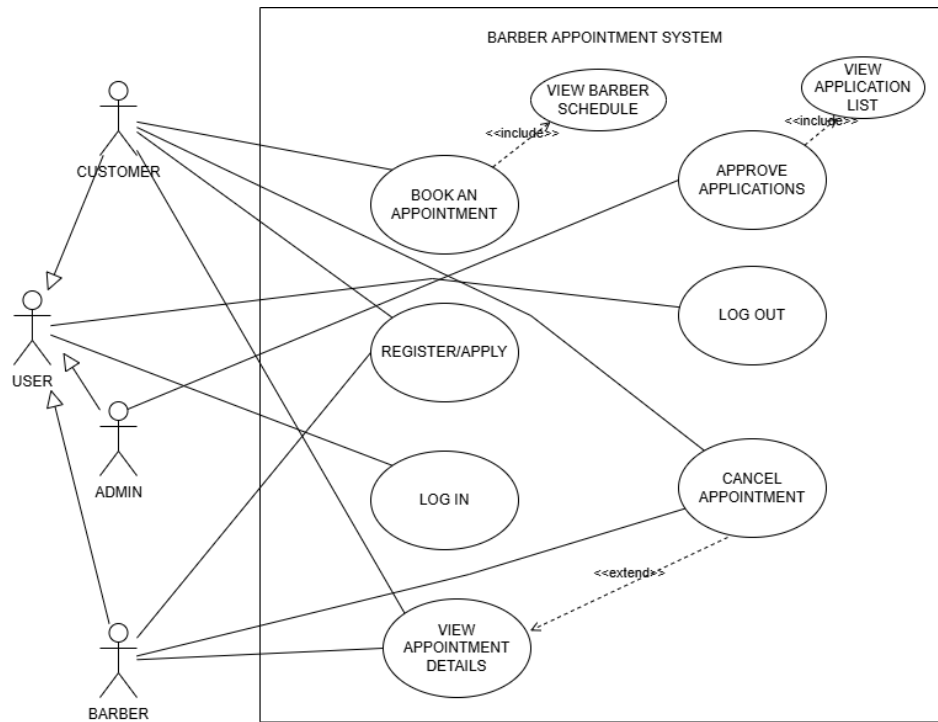


Figure 1: Use Case diagram of the system.

5.2 Class Diagrams

5.2.1 Overall System Class Diagram

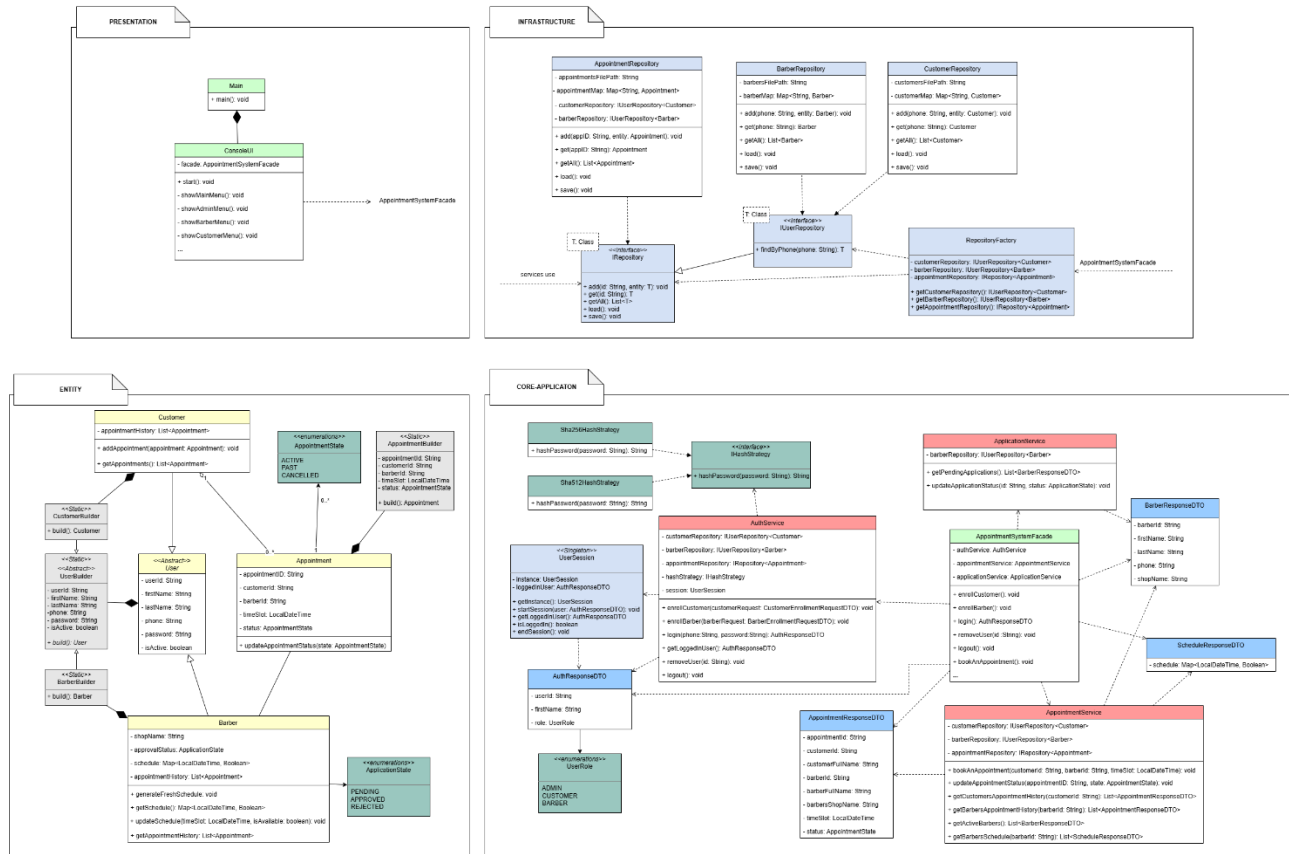


Figure 2: The complete package-level class diagram illustrating the layered architecture, including Presentation, Core Application, Infrastructure, and Entity modules.

5.2.2 Entity Layer

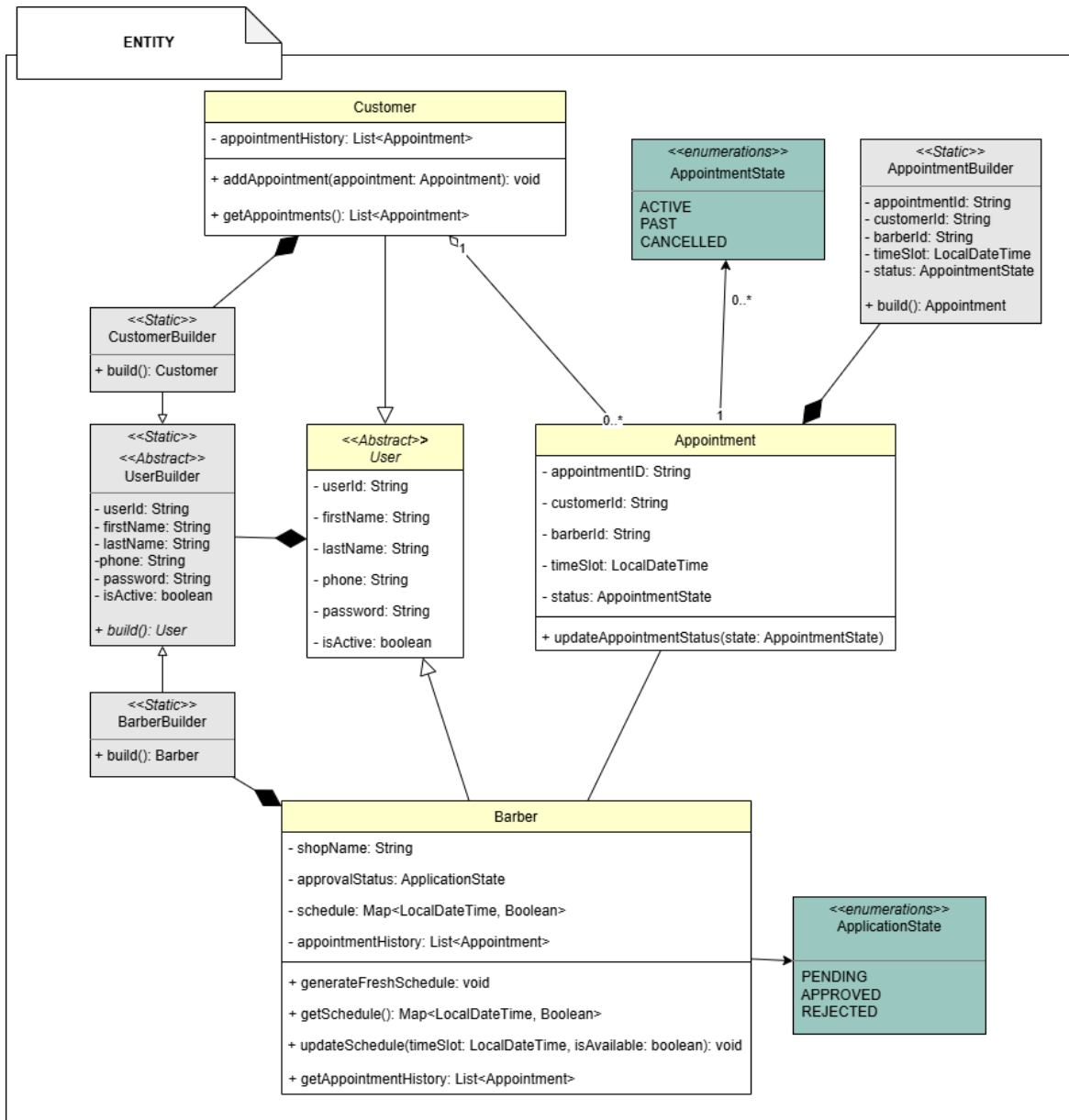


Figure 3: Detailed view of the Entity layer, highlighting the encapsulation of business states and the utilization of the Builder design pattern for object creation.

5.2.3 Infrastructure Layer

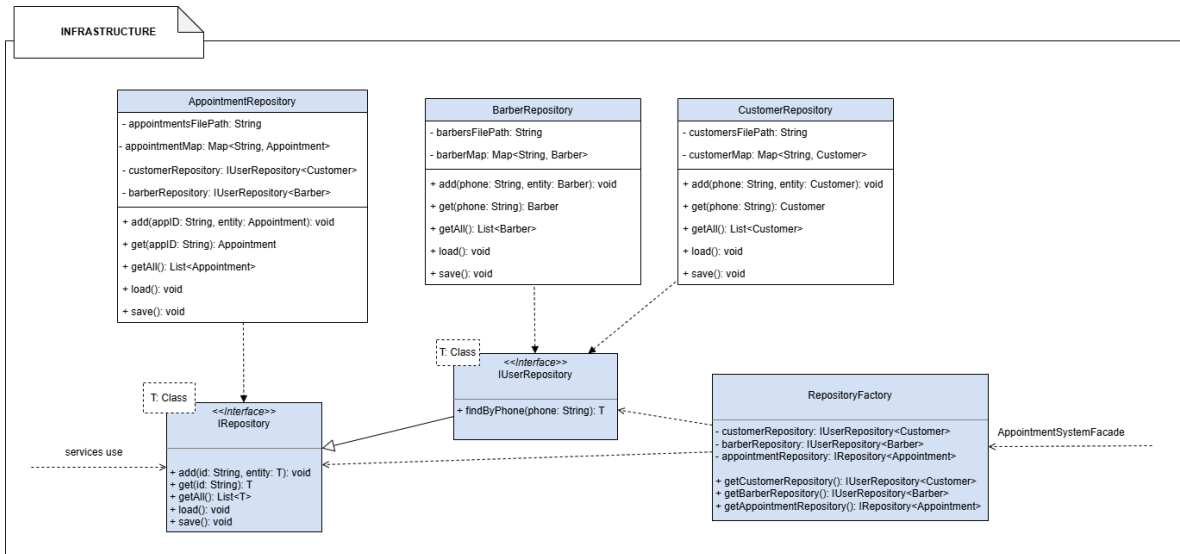


Figure 4: The Infrastructure layer showcasing the *IRepository* interface, concrete repository implementations for CSV storage, and the *RepositoryFactory* structure.

5.2.4 Presentation Layer

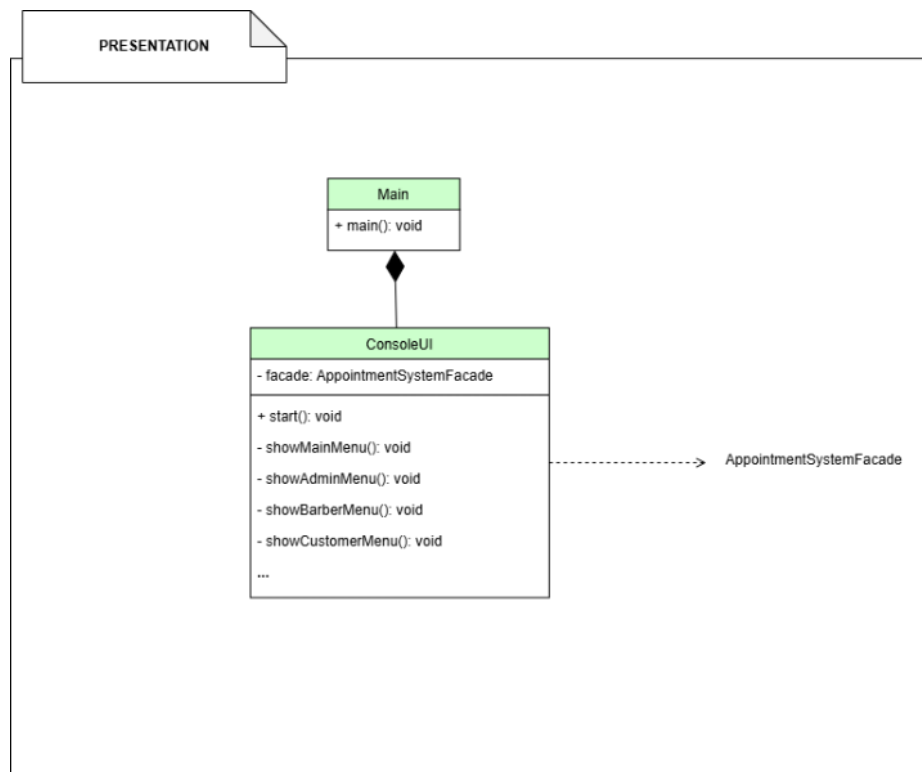


Figure 5: The Presentation layer demonstrating the "Dumb UI" approach, where the *ConsoleUI* strictly interacts with the system exclusively via the Facade and DTOs.

5.2.5 Core Application Layer

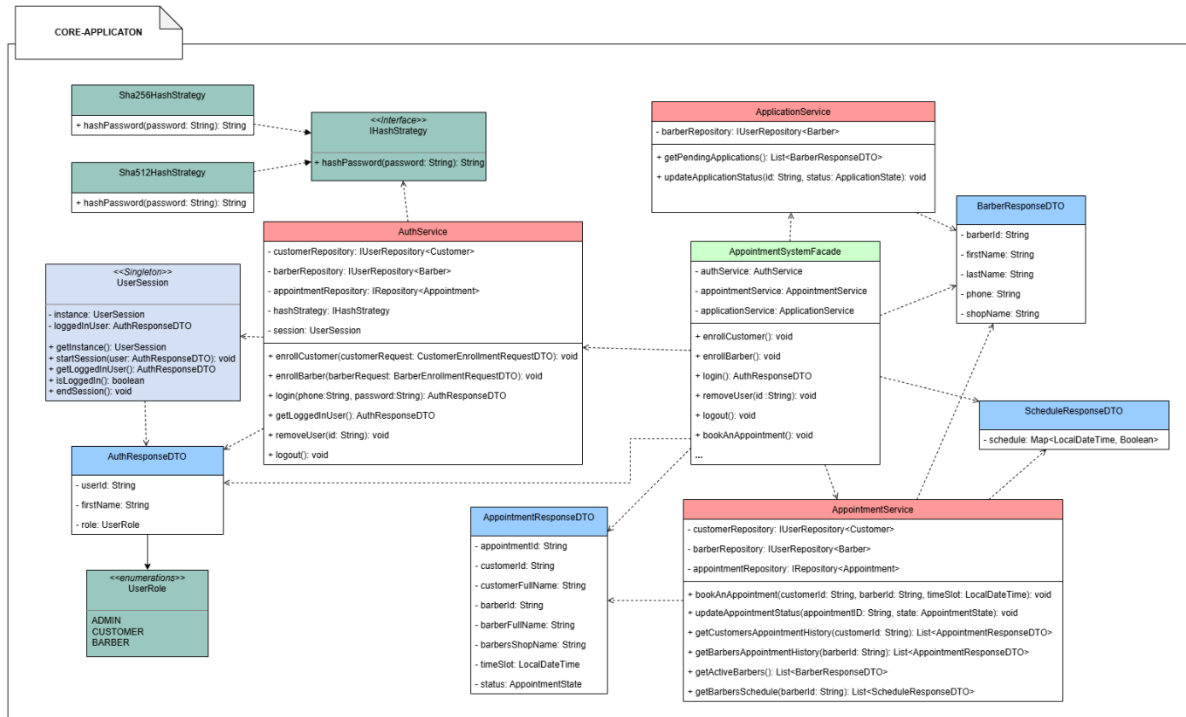


Figure 6: The Core Application layer featuring the business Service classes, the AppointmentSystemFacade acting as the architectural boundary, and the Strategy pattern implemented for secure password hashing.

5.3 Activity Diagrams

5.3.1 Admin Application Approval

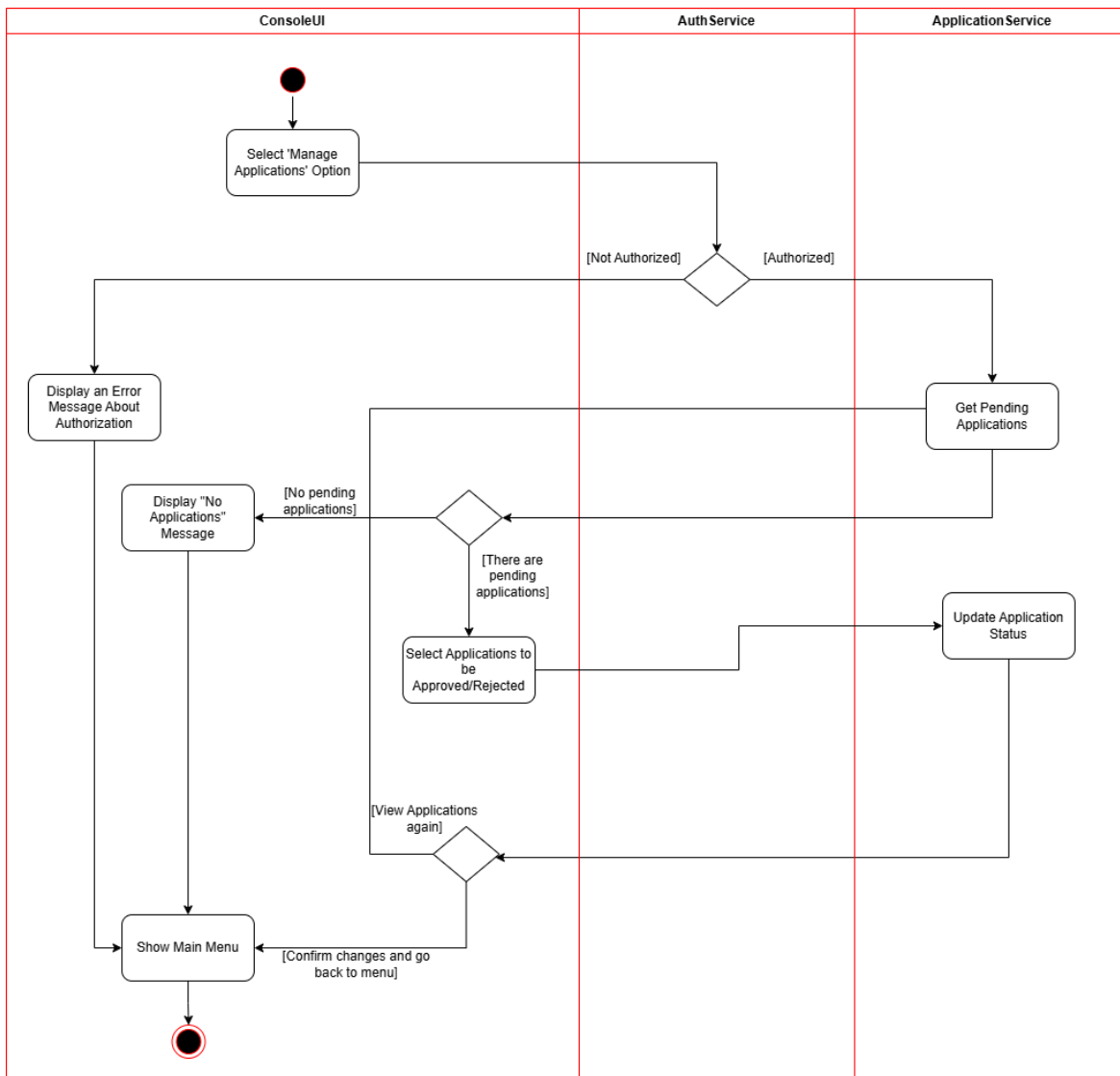


Figure 7: Admin Approval Activity Diagram

5.3.2 Appointment State Initialization Activity Diagram

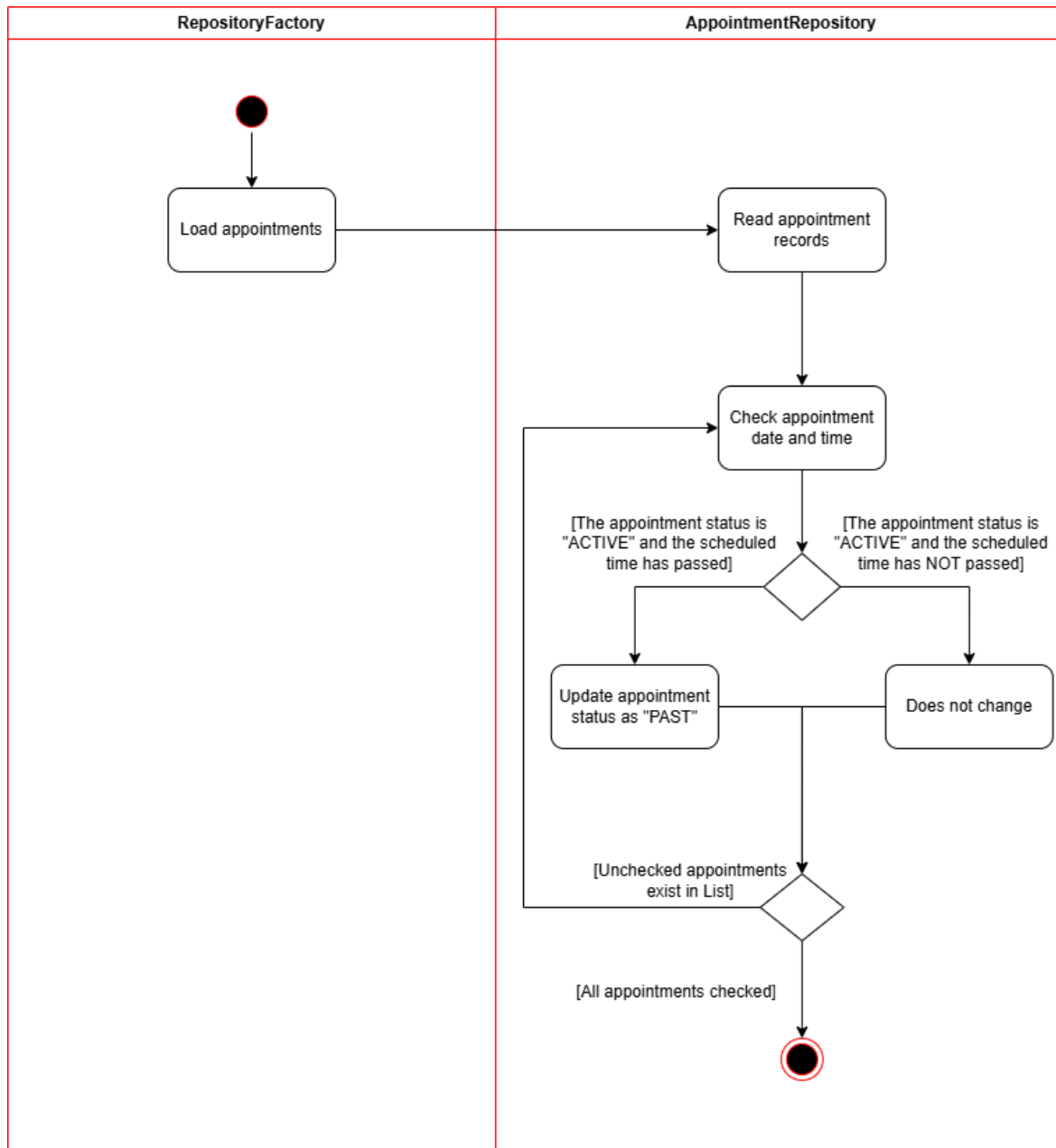


Figure 8: Activity diagram illustrating the automatic state synchronization process. It demonstrates how the system proactively updates the status of past appointments during the initial data loading phase, completely decoupled from the user interface.

5.4 Sequence Diagrams

5.4.1 Login Sequence Diagram

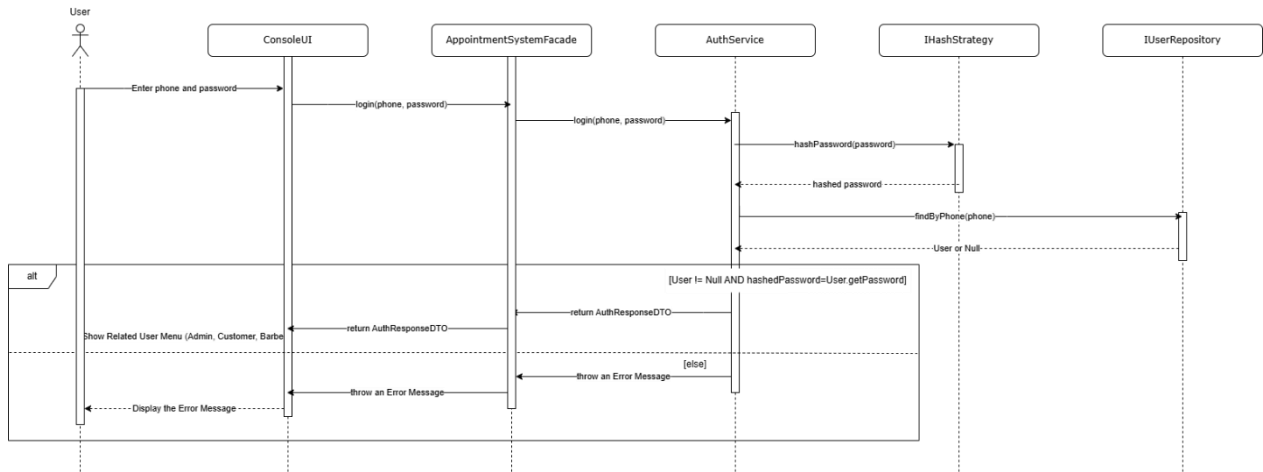


Figure 9: Sequence diagram illustrating the user authentication flow using the Facade and Strategy patterns.

5.4.2 Book An Appointment Sequence Diagram

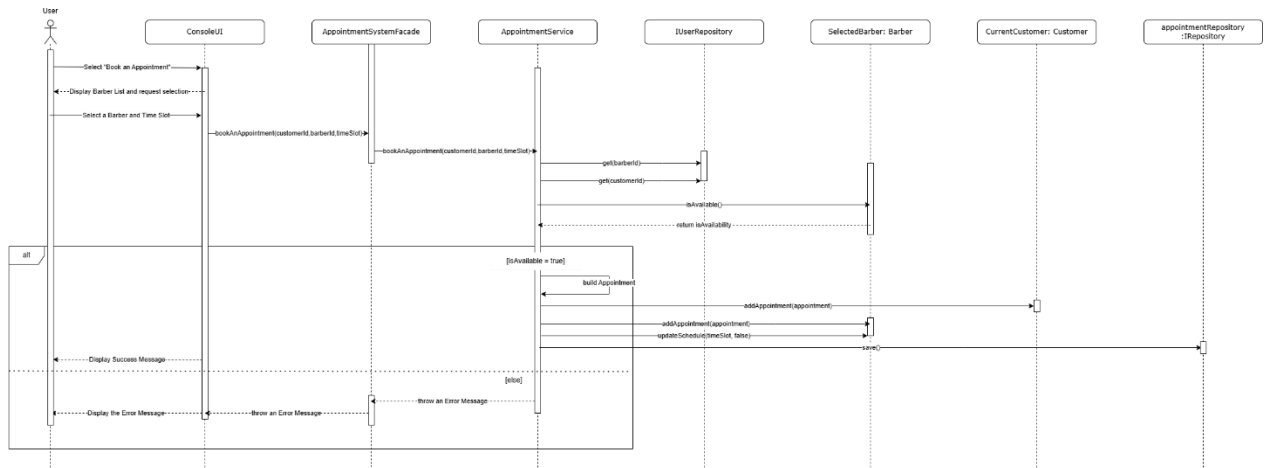


Figure 10: Sequence diagram showing the appointment booking process and bidirectional data synchronization.

5.4.3 Display Barbers Schedule Sequence Diagram

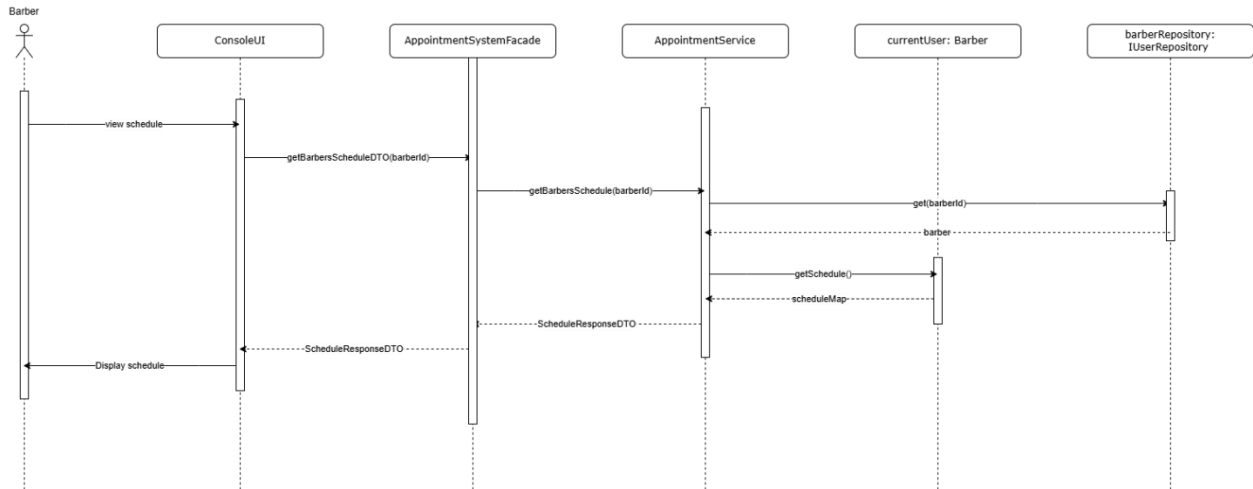


Figure 11: Sequence diagram detailing the retrieval of a barber's schedule and UI communication via DTOs.

5.4.4 Remove a User Sequence Diagram

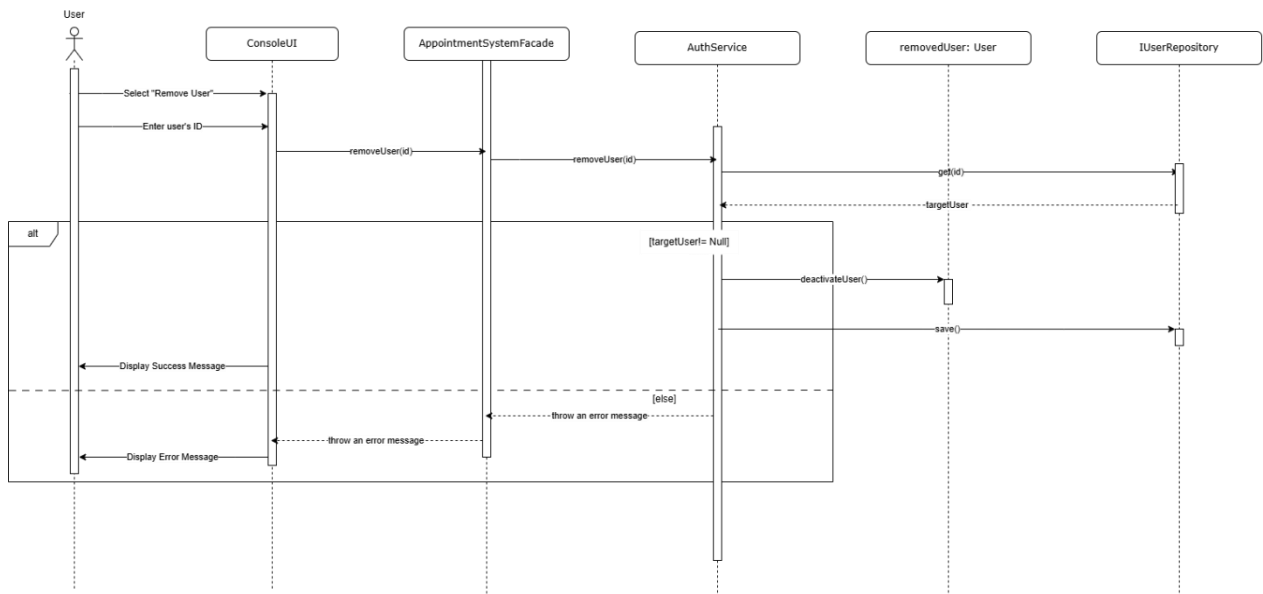


Figure 12: Sequence diagram depicting the administrative workflow for deactivating a user account and updating the repository.

5.5 Statechart Diagrams

5.5.1 Barber's Application Status Statechart Diagram

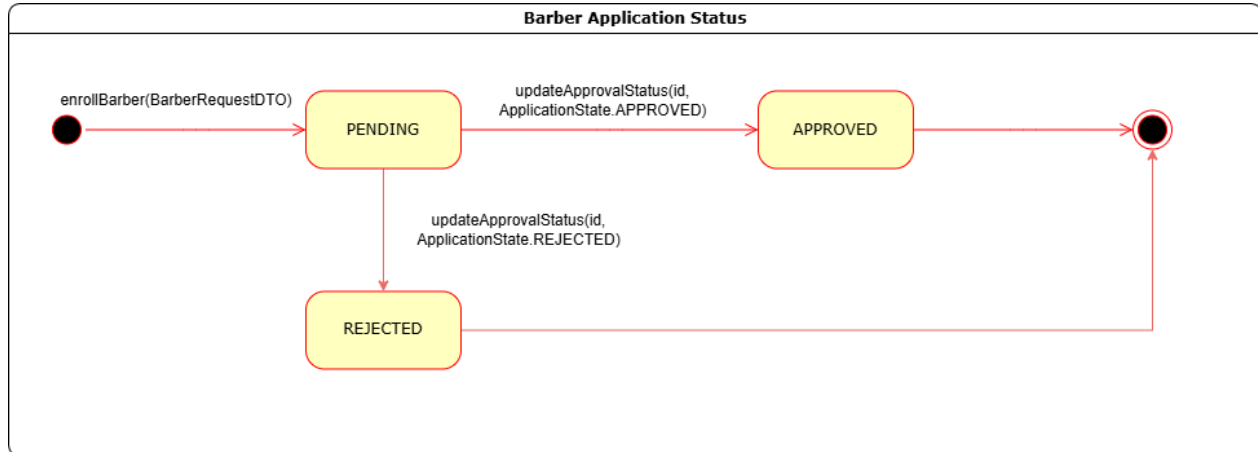


Figure 13: State machine diagram illustrating the lifecycle of a Barber's application status from initial enrollment to the final administrative decision.

5.5.2 Appointment status Statechart Diagram

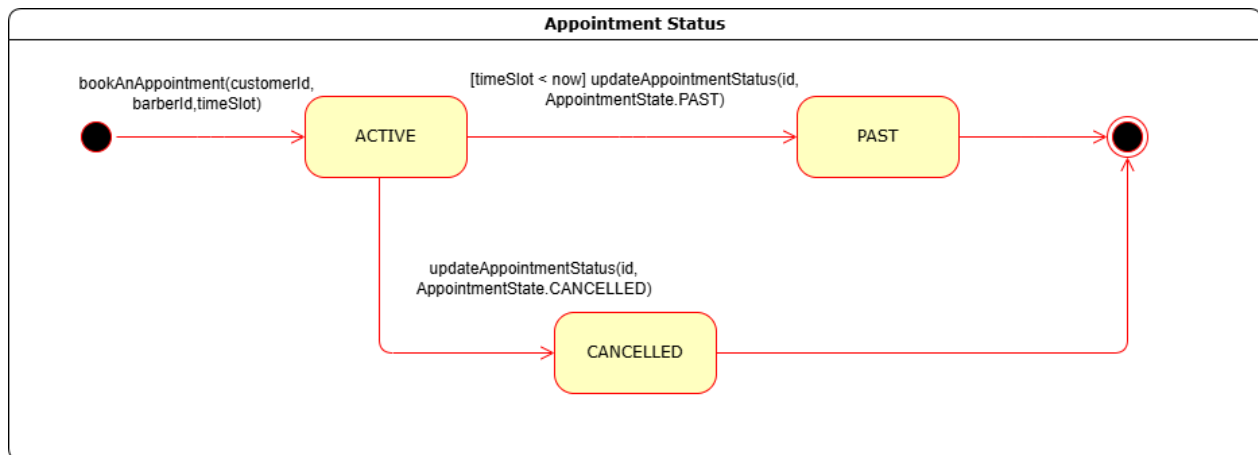


Figure 14: State machine diagram illustrating the lifecycle of an Appointment, including the automated transition to the PAST state based on time conditions.

6. DATA FLOW DIAGRAMS (DFD)

While UML diagrams illustrate the object-oriented structure and behavior of the system, Data Flow Diagrams (DFD) are utilized to map the logical flow of information. This section visualizes how data moves between external entities, system processes, and data stores without focusing on the underlying timing or object interactions.

6.1 Level 0 Context Diagram

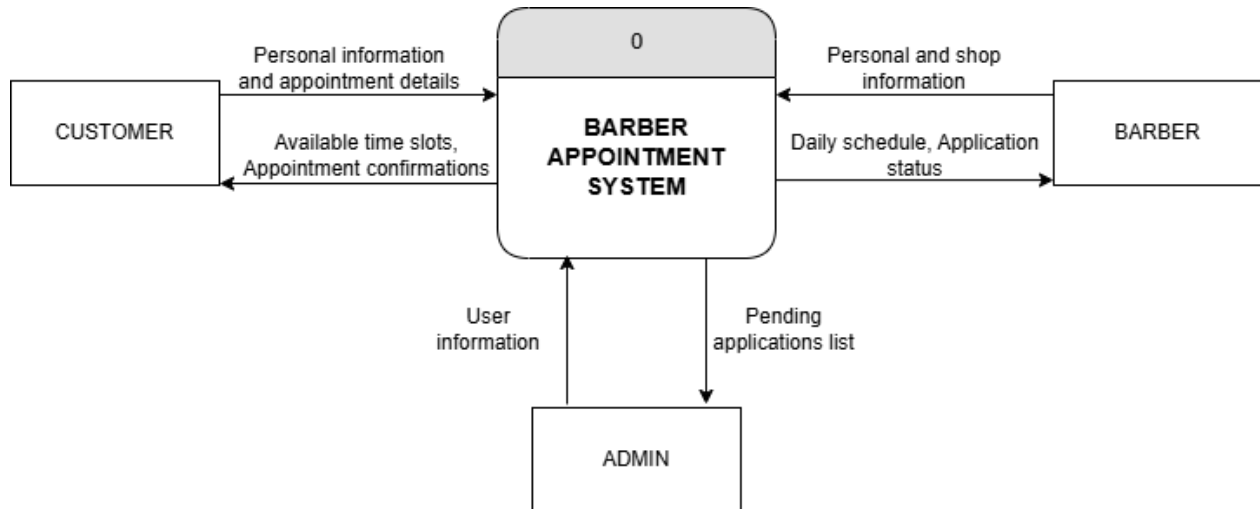


Figure 15: The Level 0 Context Diagram providing a high-level view of the Barber Appointment System and its data interactions with external entities (Customer, Barber, and Admin).

6.2 Level 1 Data Flow Diagram: Appointment Booking

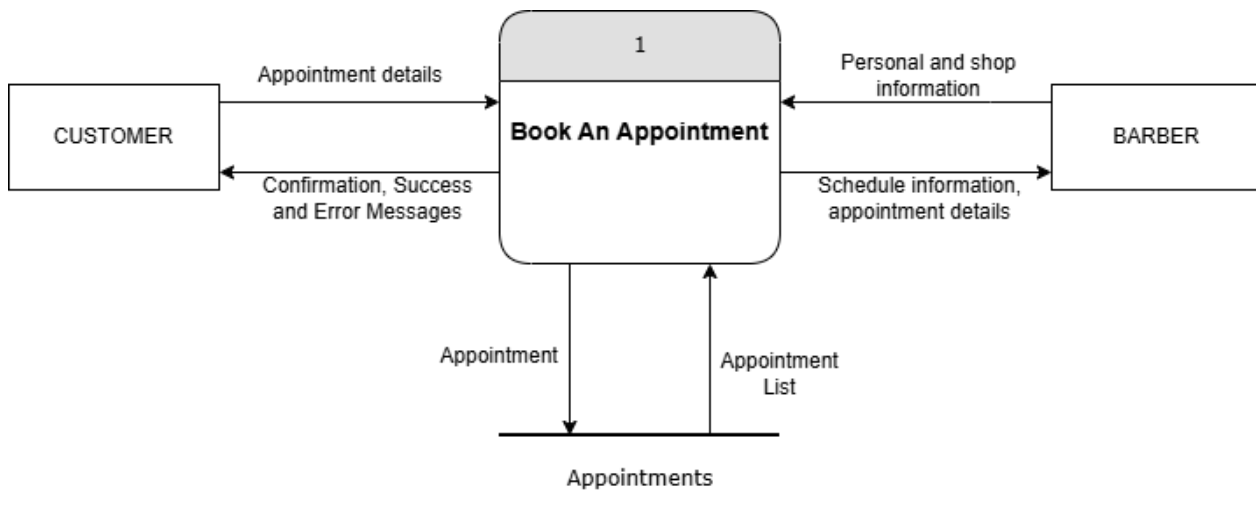


Figure 16: The Level 1 Data Flow Diagram illustrating the specific data movement, logic processing, and database interactions during the appointment booking process.

6.3 Level 1 Data Flow Diagram: User Enrollment

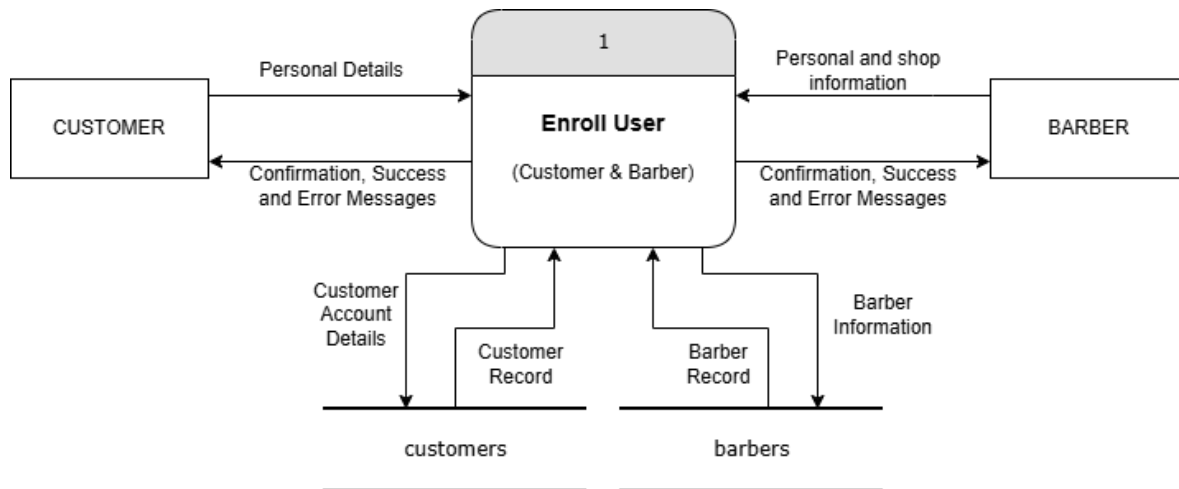


Figure 17: Level 1 Data Flow Diagram illustrating the user enrollment process. It demonstrates the data flow for both Customer and Barber registrations, including data store validation and feedback messaging.

7. DESIGN PATTERN USED

This section explains the design patterns implemented in the project. These patterns were chosen to solve specific coding problems, keep the architecture clean, and make the system easier to maintain.

7.1 Facade Pattern

- **Where it is used:** *AppointmentSystemFacade*
- **Why it is used:** The Presentation Layer (*ConsoleUI*) needs to interact with different services like *AuthService* and *AppointmentService*. Instead of allowing the UI to call these services directly and creating a complex web of connections, the Facade acts as a middleman. The UI only communicates with the Facade, which handles the complex service calls behind the scenes. This keeps the UI simple and adheres to the "dumb UI" principle.

7.2 Strategy Pattern

- **Where it is used:** *IHashStrategy* (Interface), *Sha256HashStrategy* and *Sha512Strategy*
- **Why it is used:** Storing passwords as plain text is a security risk. The system required a way to hash passwords securely. By implementing the Strategy pattern, *AuthService* does not need to know *how* the password is encrypted; it simply calls the *hashPassword()* method from the interface. If the hashing algorithm needs to be changed to *BCrypt* in the future, only a new class needs to be added without modifying the *AuthService* code.

7.3 Builder Pattern

- **Where it is used:** *AppointmentBuilder*, *CustomerBuilder*, and *BarberBuilder*
- **Why it is used:** Core domain entities have multiple variables. For example, creating a new Appointment requires a customer ID, barber ID, a specific time slot, and an initial status. Passing all these arguments into a single constructor makes the code hard to read and tends to errors. The Builder pattern allows for the step-by-step construction of these complex objects in a much cleaner and safer manner.

7.4 Factory Pattern

- **Where it is used:** *RepositoryFactory*
- **Why it is used:** The system utilizes CSV files to store Customers, Barbers, and Appointments. Instantiating repository objects directly inside the services using the new keyword would tightly couple the business logic to file operations. The *RepositoryFactory* solves this by centralizing the creation of all repositories and loading the initial data at system startup. The services simply request the required data from the factory.

7.5 Singleton Pattern

- **Where it is used:** *UserSession*
- **Why it is used:** In a console application, only one user can be logged in at any given time. There needs to be a single, shared location to store the details of the active user so that all parts of the system can verify the current user's identity. The Singleton pattern ensures that only one instance of *UserSession* is ever created, providing a global and consistent access point for the active user's data.

8. SOLID EVALUATION

8.1 Single Responsibility Principle (SRP)

The Single Responsibility Principle states that a class should have one, and only one, reason to change. In this project, responsibilities are strictly separated across different layers to prevent "God Classes" (classes that try to handle too many tasks simultaneously).

A clear example of SRP in the system is the distinction between the user interface, the business logic, and the data access mechanisms. The **ConsoleUI** class is solely responsible for capturing input and printing output. It does not calculate schedules or read files. Similarly, the **AppointmentService** is responsible for enforcing business rules (like preventing double bookings), but it does not know how data is saved to the disk. That responsibility is exclusively given to the **AppointmentRepository**.

```
public interface IRepository<T> { 14 usages 4 implementations
    /**
     * Adds a new entity instance to the repository
     * @param id The unique identifier of the instances
     * @param entity The entity instance to be added
     */
    public void add(String id, T entity); 3 implementations

    /**
     * Returns the entity instance corresponding to given id
     * @param id The unique identifier of the instance to be retrieved
     * @return The entity instance associated with the specified ID, or null if not found
     */
    public T get(String id); 19 usages 3 implementations

    /**
     * Fetches all entities from the repository and returns them as a list
     * @return A list containing all entity instance
     */
    public List<T> getAll(); 3 implementations

    /**
     * Loads entity data from the persistent storage and initializes the repository state
     */
    public void load(); 3 usages 3 implementations

    /**
     * Saves all currently existing data to the data source
     */
    public void save(); 10 usages 3 implementations
}
```

Figure 18: Repository Interface

As seen above (Figure 15), the repositories handle only file operations.

Another significant implementation of SRP is the *UserSession* class. Instead of mixing login state tracking within *the AuthService*, *UserSession* is dedicated entirely to holding the state of the currently authenticated user.

8.2 Open/Closed Principle (OCP)

The Open/Closed Principle dictates that software entities should be open for extension but closed for modification. This means new functionality should be added by writing new code rather than altering existing, working code.

The project demonstrates OCP primarily through its implementation of the Strategy Pattern for password hashing. The *AuthService* class relies on the *IHashStrategy* interface to encrypt passwords. The concrete implementation, *Sha256HashStrategy* and *Sha512HashStrategy*, provides the actual hashing algorithm.

If future security requirements dictate a shift to a stronger algorithm, such as BCrypt, the developers do not need to modify the *AuthService* class. They only need to create a new *BcryptHashStrategy* class that implements *IHashStrategy* and injects it into the system. The existing authentication logic remains completely untouched.

```
public interface IHashStrategy { 9 usages 2 implementations
    /**
     * Hashes the given raw password to enhance the security.
     * @param rawPassword The raw password to be hashed.
     * @return The hashed password.
     */
    String hashPassword(String rawPassword); 3 usages 2 implementations
}
```

Figure 19: Hash Strategy Interface

```

public class Sha256HashStrategy implements IHashStrategy { 4 usages

    /**
     * Hashes the raw password based on SHA-256 cryptographic algorithm.
     * @param rawPassword The raw password to be hashed.
     * @return The hashed password represented as a hexadecimal string.
     * @throws RuntimeException if the SHA-256 digest algorithm is not found.
     */
    public String hashPassword(String rawPassword){...}
}

```

Figure 20: A concrete strategy class which implements the IHashStrategy Interface.

8.3 Liskov Substitution Principle (LSP)

The Liskov Substitution Principle ensures that objects of a superclass shall be replaceable with objects of its subclasses without breaking the application. Subclasses must honor the contracts defined by their base classes.

In the entity layer, this principle is highly visible. The system defines an abstract base class *User*, which contains fundamental attributes like ID, name, phone, and password. The *Customer* and *Barber* classes inherit from *User*.

Throughout the core application, particularly in the authentication and repository layers, methods are designed to operate on the generic *User* type. Because *Customer* and *Barber* do not override the base methods in a way that alters their expected behavior, they can safely be substituted wherever a *User* is expected. This allows the *AuthService* to authenticate both customers and barbers using the exact same logic.

8.4 Interface Segregation Principle (ISP)

The Interface Segregation Principle states that no client should be forced to depend on methods it does not use. Large, general-purpose interfaces should be split into smaller, more specific ones.

Instead of creating a massive *IGlobalRepository* interface that forces every data class to implement unnecessary methods (e.g., forcing a *Barber* repository to implement appointment-specific search methods), the system uses focused interfaces. The base *IRepository<T>* contains only the fundamental CRUD (Create, Read, Update, Delete) operations.

When specific entities require specific queries, the interfaces are segregated. For instance, *IUserRepository* extends *IRepository* to add user-specific behaviors (such as *findByPhone()*), ensuring that the *AppointmentRepository* is not responsible for user-related query methods.

```
public interface IUserRepository<T> extends IRepository<T> { 18 usages 2 implementations

    /**
     * Searches the repository for an entity with the specified phone number.
     * @param phone The phone number to search for
     * @return The entity if found, or null if no matching phone number exists
     */
    T findByPhone(String phone); 4 usages 2 implementations
}
```

Figure 21: User Repository Interface containing user specific *findByPhone* method and implements general repository interface.

8.5 Dependency Inversion Principle (DIP)

The Dependency Inversion Principle is perhaps the most critical structural rule in this architecture. It states that high-level modules should not depend on low-level modules; both should depend on abstractions. Furthermore, abstractions should not depend on details.

This principle is strictly enforced between the *Core Application* layer and the *Infrastructure* layer. The *AuthService* (a high-level module) does not instantiate the concrete *CustomerRepository* (a low-level file I/O module) directly using the `new` keyword. If it did, the business logic would be tightly coupled to CSV file storage.

Instead, *AuthService* depends on the *IUserRepository* abstraction. The actual concrete repository instances are created at the application's root (inside the *RepositoryFactory*) and are passed into the services via Constructor Injection. This manual dependency injection ensures that database technology can be entirely replaced (e.g., moving from CSV to MySQL) without altering a single line of code in the Service layer.

```

/**
 * Initializes the repository dependencies.
 * @param customerRepository The repository contract containing all customers in the system.
 * @param barberRepository The repository contract containing all barbers in the system.
 * @param appointmentRepository The repository contract containing all appointments in the system.
 * @param hashStrategy The hashing strategy which the raw passwords will be hashed based on it (e.g, SHA-256, SHA-512).
 * @param session The session object
 */
public AuthService(IUserRepository<Customer> customerRepository, IUserRepository<Barber> barberRepository, 2 usages
    IRepository<Appointment> appointmentRepository ,IHashStrategy hashStrategy, UserSession session){
    this.customerRepository = customerRepository;
    this.barberRepository = barberRepository;
    this.appointmentRepository = appointmentRepository;
    this.hashStrategy = hashStrategy;
    this.session = session;
}

```

Figure 22: Constructor method of the AuthService class. It depends on repository contracts rather than the low level (repository) classes.

9. TESTING STRATEGY

The testing strategy for the Barber Appointment System is designed to verify the correctness of the core business rules, validate user input handling, and ensure that data is safely stored and retrieved from the CSV files. Because the system strictly follows a Layered Architecture and utilizes Dependency Injection, the testing process is highly structured and manageable.

The testing approach is categorized into four main areas:

9.1 Unit Testing (Business Logic Isolation)

Due to the Dependency Inversion Principle, the Core Application layer is completely isolated from the file system. This allows the business logic to be tested independently. For example, the *AppointmentService* can be tested by injecting fake (mock) repository objects via its constructor. This makes it possible to verify critical business rules—such as preventing a double-booking for the same time slot or ensuring that past appointments cannot be booked—without needing to read or write to actual CSV files.

9.2 Data Persistence and Integration Testing

This phase focuses on the infrastructure layer and ensuring data integrity. The primary goal is to verify the bidirectional conversion between Java objects (Entities) and CSV text formatting. Tests in this category verify that:

- The *RepositoryFactory* correctly initializes and loads all data into memory during system startup.
- When an appointment's status is updated (e.g., changing from "ACTIVE" to "CANCELLED" or "PAST"), the system correctly overwrites the specific row in the CSV file without corrupting the rest of the data.
- The system handles file-related exceptions gracefully (e.g., creating a new, empty CSV file if the expected file is missing upon startup).

9.3 Input Validation and Edge Case Handling

Since the Presentation Layer is a console-based interface, it is highly susceptible to incorrect user inputs. The system is tested against various edge cases to prevent runtime crashes. Key validation tests include:

- **Format Mismatches:** Entering letters when the menu expects a numerical option.

- **Duplicate Data:** Attempting to register a new user with a phone number that already exists in the system.
- **Boundary Conditions:** Attempting to select a time slot outside of a barber's working hours. The system is expected to catch these errors and return meaningful error messages to the *ConsoleUI* without terminating the application unexpectedly.

9.4 End-to-End (E2E) Workflow Testing

Finally, manual scenario-based testing is conducted to simulate the complete lifecycle of the application across different user roles. A standard E2E test scenario includes:

1. A new Barber register (Status: PENDING).
2. The Admin logs in, reviews the application, and approves it.
3. A Customer logs in, views the newly approved Barber, and books a time slot.
4. The system is restarted to verify that the booking was successfully saved and reloaded. This ensures that the *AppointmentSystemFacade* correctly orchestrates the communication between all underlying services from a user's perspective.

10. LIMITATIONS

While the Barber Appointment System is architecturally sound and successfully meets its functional requirements, there are inherent limitations based on the chosen technology stack and the scope of a console-based application. Recognizing these limitations is crucial for understanding the system's operational boundaries.

10.1 Data Persistence and Concurrency

The system utilizes CSV files for data storage. While this is sufficient for a localized, single-instance application, it presents limitations regarding scalability and concurrency.

- **Lack of ACID Properties:** Unlike a relational database management system (RDBMS), CSV files do not provide built-in transaction management. In a highly concurrent environment where multiple users attempt to book the same time slot at the exact same millisecond, file-level locking mechanisms would be required to prevent data corruption or race conditions.
- **Performance Overhead:** As the number of users and past appointments grows, reading and parsing the entire CSV file into memory during system startup will eventually become a performance bottleneck.

10.2 User Interface Constraints

The application relies on *ConsoleUI*. While the "dumb UI" principle keeps the architecture clean, the terminal environment inherently limits the user experience (UX).

- **Usability:** Customers cannot visually interact with a calendar to select time slots. They are required to navigate through sequential text menus and input data strictly in the expected formats.
- **Accessibility:** The lack of a Graphical User Interface (GUI) or a web-based frontend makes the application unsuitable for the average, non-technical end-user in a real-world commercial setting.

10.3 Security and Session Management

Although the system implements the Strategy pattern for password hashing, it is a local console application, which limits its security scope.

- **Local Execution:** The *UserSession* singleton works perfectly for a single instance running on one machine. However, if this were a distributed client-server application, session management would require tokens (e.g., JWT) and secure network protocols (HTTPS) to prevent session hijacking and ensure secure data transmission.

10.4 Functional Scope Limitations

The current iteration of the system is strictly focused on demonstrating **clean architecture** through the core "happy-path" workflows (e.g., registering, approving, and booking). As a result, several standard quality-of-life features are excluded from the current scope.

- **Feature Absence:** Functionalities such as password recovery ("forgot password"), dynamic user profile editing, or directly modifying an existing appointment (which currently requires a cancellation and re-booking) are not implemented. While these were considered out-of-scope for this foundational architectural phase, they would be strictly necessary for a production-ready application.

11. Reflection and Future Improvements

The development of the Barber Appointment System provided significant insights into software architecture and object-oriented design. Transitioning from theoretical concepts to a fully implemented Layered Architecture presented both challenges and valuable learning opportunities.

11.1 Reflection and Lessons Learned

One of the primary challenges was strictly obeying the SOLID principles and maintaining a clean architecture without the assistance of modern frameworks (such as Spring Boot). Implementing manual Dependency Injection and ensuring that the UI remained completely decoupled from the data access layer required careful planning. The biggest lesson learned during this implementation phase was the power of abstractions. By utilizing interfaces (e.g., *IRepository*, *IHashStrategy*) and design patterns (like Facade and Factory), it became evident how easily a complex system can be managed, tested, and extended when components are not tightly coupled.

11.2 Future Enhancements

While the current system fulfills its foundational requirements, several enhancements can be introduced to elevate it to a production-ready application.

- **Backend & Functional Expansions:** The immediate next step is to expand the functional scope by introducing standard quality-of-life features. This includes implementing secure password recovery ("forgot password" flow), allowing users to dynamically edit their profile information, and adding the ability to modify an existing appointment directly rather than requiring a cancellation and re-booking.
- **Frontend & User Experience (GUI):** The current *ConsoleUI* limits user interaction. Because the system utilizes the Facade pattern, the terminal interface can be smoothly detached and replaced with a Graphical User Interface (GUI), such as JavaFX or an API-driven web frontend, without altering the underlying business logic.
- **Database Migration:** Currently, the system relies on CSV files for data persistence. To handle higher concurrency and ensure data integrity under heavy loads, a major future improvement would be migrating the infrastructure layer to a Relational Database Management System (RDBMS) using SQL. Thanks to the Dependency Inversion Principle, this transition would simply require creating new SQL-based repository classes and injecting them via the existing *RepositoryFactory*.

12. CONCLUSION

The Barber Appointment System highlights the application of modern software engineering practices in the development of a flexible and well-structured application. The goal of the project was not just to build a simple appointment scheduler, but to create a system with a clear architecture that supports long-term maintenance and future improvements.

During development, **SOLID** principles were carefully followed, and several design patterns such as Factory, Builder, Facade, and Strategy were implemented. These approaches improved code organization and established a clear distinction between the Presentation, Core Application, and Infrastructure layers. Because of this layered structure, the main business operations remain independent from the user interface and data storage components.

At present, the application runs through a command-line interface and stores data using CSV files. However, the system design allows new technologies to be added easily in the future. For example, a graphical interface or a relational database can be integrated without major changes to the existing business logic. In conclusion, the project demonstrates how clean and adaptable software architecture can be developed in Java using industry-standard design principles.

13. REFERENCES

1. Term_Project_Definition.pdf, Available on SAKAI System.
2. Object-Oriented Analysis and Design, Brahma Dathan & Sarnath Ramnath, “2.7.2 Use cases and use case diagrams.”
3. Provided Lecture Sources on Sakai System.
4. Draw.io, Creating UML Diagram, Available: <https://www.drawio.com/> [Accessed: March 2026]
5. Refactoring Guru, Design Patterns, Available: <https://refactoring.guru/design-patterns> [Accessed: May 2026]

APPENDIX A: USER MANUEL & SYSTEM EXECUTION

A.1 Main Menu

```
--- BARBER APPOINTMENT SYSTEM ---  
1-Login  
2-Sign up as a Customer  
3-Apply to system as a Barber  
0-Exit  
Please select an operation.  
Input:
```

Figure 23: The initial start-up screen of the Barber Appointment System.

Upon launching the application, the system presents the main navigation menu. The user is prompted to input a numeric value to select one of the following operations:

- **1-Login:** Directs the user to the authentication module. Based on the provided credentials, the *UserSession* is established, and the system automatically routes the user to the appropriate dashboard (Customer, Barber, or Admin).
- **2-Sign up as a Customer:** Initiates the customer registration flow. The user is prompted to enter personal details, which are immediately saved to the data store as an active customer.
- **3-Apply to system as a Barber:** Starts the enrollment process for a new barber. The user enters their professional details, and the system creates a new record with a PENDING status, which requires administrative approval before logging in.
- **0-Exit:** Safely terminates the application loop and closes the system.

A.2 Login and Customer Menu

```
-- Login --

Please enter your phone number. (0 for main menu)
Input: 05329920001
Please enter your password. (0 for main menu)
Input: ugur123
SUCCESS: Login is successful. Welcome Uğurcan

-- Customer Menu --
1-Book an Appointment
2-View Active Appointments
3-View Appointment History
4-Cancel Appointment
5-Remove My Account
0-Logout
Please select a menu.
Input:
```

Figure 24: The authentication process and the subsequent Customer Dashboard.

When the user selects the Login option from the main menu, the system requests their registered phone number and password. The application verifies these credentials securely.

Admin Access Note: The system includes a built-in administrative account. To access the Admin Dashboard, the user must input admin in the phone number field and 123 as the password.

Upon successful authentication, the singleton *UserSession* is initialized, and the system routes the user to their respective role-based dashboard. In the example above, a customer successfully logs in and is presented with the **Customer Menu**, which includes the following operations:

- **1-Book an Appointment:** Initiates the process of selecting a barber and securing an available time slot.
- **2-View Active Appointments:** Displays the customer's upcoming and valid appointments.

- **3-View Appointment History:** Shows a historical log of past or cancelled appointments.
- **4-Cancel Appointment:** Allows the customer to select a currently active appointment and change its state to cancelled.
- **5-Remove My Account:** Initiates the account deactivation (soft delete) process for the logged-in customer.
- **0-Logout:** Clears the active session and returns the user to the initial Main Menu.

A.3 Booking an Appointment: Barber Selection

```
- Book An Appointment -
1) -Barber ID: b-be3996e6      - Barber Name: Can Orhan      - Shop Name: Simotcan      - Phone: 55555
2) -Barber ID: b-5a646604      - Barber Name: Gizem Yalin    - Shop Name: Gizem Beauty   - Phone: 05442327021
3) -Barber ID: b-da139ce9      - Barber Name: Metin Meral    - Shop Name: Hair Designer Metin - Phone: 05457412235
4) -Barber ID: b-100f7e10      - Barber Name: Mehmet Ali Kara - Shop Name: Barber Mehmet Ali - Phone: 05524892190
5) -Barber ID: b-85e2cacd      - Barber Name: Melih Çelik    - Shop Name: Melih's Barber Shop - Phone: 05422216766

Please enter the index of the barber. (0 for Customer Menu)
Input: 3
```

Figure 25: The barber selection interface displaying all approved and active barbers in the system.

When a customer initiates the "Book an Appointment" process, the system queries the underlying repository and retrieves a formatted list of all available barbers. To provide a clear and user-friendly experience, the console displays essential information for each professional, including their unique Barber ID, full name, shop name, and contact number.

The user is prompted to input the index number corresponding to their preferred barber. For instance, entering 3 selects the barber at that specific index and proceeds to the time slot selection phase. The application also provides a safe-exit mechanism, allowing the user to input 0 to cancel the operation and seamlessly return to the Customer Menu without triggering any unintended state changes.

A.4 Booking an Appointment: Schedule and Time Slot Selection

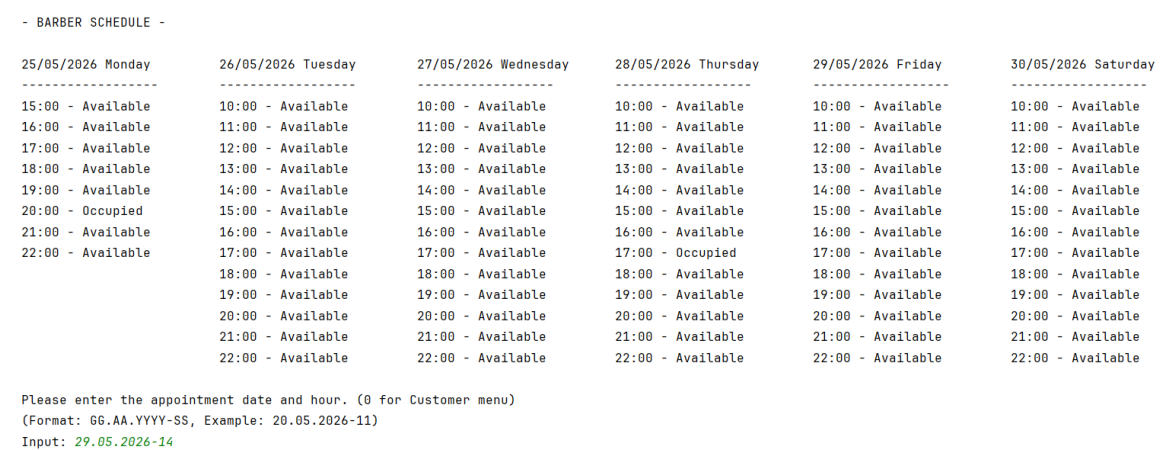


Figure 26: The dynamic barber schedule matrix displaying available and occupied time slots for a six-day period.

After a specific barber is selected, the system retrieves their personalized schedule and renders it as a structured, multi-column grid in the console. This interface allows the customer to visually inspect the barber's availability across a six-day rolling window.

Several critical engineering implementations are visible in this process:

- **Dynamic Time Filtering:** The system evaluates the current system time and automatically filters out past hours for the current day. This prevents users from booking appointments in the past.
- **State Representation:** Each hourly slot accurately reflects the data stored in the repository, clearly marked as either Available or Occupied to prevent double-booking attempts.
- **Format-Strict Input Handling:** The system prompts the user to enter the desired slot using a strict formatting pattern (DD.MM.YYYY-HH). This enforces data consistency before the input is parsed by the backend services, minimizing runtime exceptions.

In the provided example, the user inputs 29.05.2026-14 to secure the 14:00 time slot on Friday, which will trigger the final confirmation and synchronization step in the *AppointmentService*.

A.5 Booking an Appointment: Confirmation and Finalization

```
Please confirm your appointment details.  
Appointment Details:  
Date: 29.05.2026 14:00  
Barber Details:  
Barber Name:Metin Meral  
Shop Name: Hair Designer Metin  
  
1-Confirm  
2-Edit (Starting from the beginning)  
0-Cancel and go back to menu  
Input: 1  
SUCCESS: Appointment is created successfully.  
You can view your appointment details in the 'View Active Appointments' menu.
```

Figure 27: The appointment confirmation screen requiring final user validation before database synchronization.

Before permanently saving the transaction, the application introduces a confirmation checkpoint. This is a critical User Experience (UX) feature designed to prevent accidental or incorrect bookings. The system aggregates the previously selected data (specifically the parsed date, time, and the barber's details) and presents a final summary to the customer.

At this stage, the user is provided with three distinct paths:

- **1-Confirm:** Approves the transaction. Upon this input, the *AppointmentService* utilizes the internal Builder pattern to instantiate the new Appointment entity. It then executes the bidirectional synchronization, updating both the customer's and barber's schedules, and finally writes the changes to the CSV data store.
- **2-Edit (Starting from the beginning):** Discards the current temporary selections and restarts the booking workflow from the barber selection phase, allowing the user to correct any mistakes.
- **0-Cancel and go back to menu:** Completely aborts the operation and safely returns the user to the Customer Menu without creating any records.

In the provided screenshot, the user inputs 1. The system successfully processes the transaction and returns a positive feedback message (SUCCESS), informing the user that the details can now be tracked via the 'View Active Appointments' module.

A.6 Viewing Active Appointments

-Active Appointments-

Appointment ID : app-ab6eee16
Customer Name : Uğurcan Çakır
Barber Id : b-100f7e10
Barber Name : Mehmet Ali Kara
Shop Name : Barber Mehmet Ali
Appointment Date : 25.05.2026 17:00
Appointment Status : ACTIVE

Appointment ID : app-ec6032e2
Customer Name : Uğurcan Çakır
Barber Id : b-da139ce9
Barber Name : Metin Meral
Shop Name : Hair Designer Metin
Appointment Date : 28.05.2026 17:00
Appointment Status : ACTIVE

Appointment ID : app-60eebc82
Customer Name : Uğurcan Çakır
Barber Id : b-da139ce9
Barber Name : Metin Meral
Shop Name : Hair Designer Metin
Appointment Date : 29.05.2026 14:00
Appointment Status : ACTIVE

Figure 28: The active appointments interface displaying all upcoming and valid bookings for the authenticated customer.

By selecting the 'View Active Appointments' option, the system uses the *UserSession* to query the *AppointmentRepository* and retrieve only the customer's ACTIVE records. The console then formats this data into readable blocks containing the unique Appointment ID, barber details, and exact time. As demonstrated, the newly created May 29th appointment is successfully listed, verifying the complete end-to-end data persistence and retrieval flow.

APPENDIX B: Data Persistence AND CSV File Structure

The system relies on flat-file storage for data persistence, utilizing three primary CSV (Comma-Separated Values) files: `customers.csv`, `barbers.csv`, and `appointments.csv`.

For the application to read and write data successfully, these files must be located directly in the root directory of the project (accessed directly by their filenames, e.g., `"barbers.csv"`).

To ensure accurate data parsing and prevent issues with **user inputs that might contain commas**, the system strictly utilizes a semicolon (;) as the data delimiter.

Additionally, the first row of every file is strictly reserved for the column headers. The underlying *RepositoryFactory* uses these formats to map the text data into their corresponding Java entities.

The required header structures for each file are as follows:

- **Customer File (`customers.csv`):**
customerId;firstName;lastName;phone;password;isActive
- **Barber File (`barbers.csv`):**
barberId;firstName;lastName;phone;password;isActive;shopName;approvalStatus
- **Appointment File (`appointments.csv`):**
appointmentId;customerId;barberId;timeSlot;status